DATA ENGINEERING / HANDS-ON LAB

Retail Sales Data Warehouse

Student assignment guide | SQL Server + SSIS | Staging, Dimensions & Fact Table

Mission

Design and create an analytical data warehouse called **RetailDWH** using the supplied SQL Server operational database and product catalog file. Build the **staging layer**, four **dimension tables**, and a **sales fact table**. Then explain how an SSIS pipeline would populate them.

Important: This is a student task brief, not a ready-made SQL solution. Create the DWH objects yourself; do not copy source OLTP tables directly into the star schema.

Learning outcomes

- Distinguish operational source tables, staging tables, dimensions, and facts.
- Choose the correct grain, business keys, surrogate keys, and foreign-key relationships.
- Design SCD Type 1 dimensions and an SCD Type 2 product dimension.
- Define sales measures and validate data quality before loading the fact.
- Draw and explain the end-to-end data flow and star schema.

Provided inputs

File	Purpose
01_Create_OLTP_Source(1).sql	Creates RetailOLTP with Customers, Stores, Orders, OrderLines.
01b_Insert_OLTP_Sample_Data(1).sql	Populates the operational source with sample transactions.
ProductCatalog_Batch1.csv	Product master data (24 products) outside RetailOLTP.

Suggested tools: SQL Server Management Studio (SSMS), SQL Server, Visual Studio / SSDT with SSIS. The diagrams on the next pages show the **target design** students should implement.

RETAIL SALES DATA WAREHOUSE | FULL PIPELINE

Source systems > Staging > SSIS transformation > RetailDWH star schema

01 SOURCE SYSTEMS

SQL Server | RetailOLTP

dbo.Customers
Customer master

dbo.Stores
Store master

dbo.Orders
Order header

dbo.OrderLines
Sales order detail

CSV | ProductCatalog_Batch1.csv
Product master outside RetailOLTP

02 STAGING (stg)

stg.Customers
CustomerID, CustomerName, City, Region, Email, LastModifiedAt

stg.Stores
StoreID, StoreName, City, Region, LastModifiedAt

stg.Orders
OrderID, CustomerID, StoreID, OrderDate, OrderStatus, LastModifiedAt

stg.OrderLines
OrderLineID, OrderID, ProductID, Quantity, UnitPrice, DiscountAmount, LastModifiedAt

stg.ProductCatalog
ProductID, SKU, ProductName, Category, Brand, UnitCost, IsActive, LastModifiedAt

03 ETL / SSIS

1. Validate & clean
Types, NULLs, duplicates

2. Load dimensions
SCD Type 1 and Type 2

3. Generate calendar
DateKey, Month, Quarter, Year

4. Resolve surrogate keys
CustomerKey / StoreKey / ProductKey

5. Calculate sales measures
GrossSales and NetSales

6. Load fact
One row per OrderLineID

04 RETAILDWH

dim.DimDate
Calendar attributes derived from date

dim.DimCustomer
CustomerKey + SCD Type 1

dim.DimStore
StoreKey + SCD Type 1

dim.DimProduct
ProductKey + SCD Type 2 history

fact.FactSales
Four FKs + OrderID + sales measures

Warehouse tables start EMPTY; SSIS loads them.

FINAL WAREHOUSE TABLES | COMPLETE COLUMN LISTS

The columns below are the target DWH design, not necessarily columns physically present in the OLTP source.

dim.DimDate

DateKey (PK)
FullDate (UNIQUE)
DayNumber
MonthNumber
MonthName
QuarterNumber
YearNumber

dim.DimCustomer

CustomerKey (PK)
CustomerID (UNIQUE)
CustomerName
City
Region
Email

dim.DimStore

StoreKey (PK)
StoreID (UNIQUE)
StoreName
City
Region

dim.DimProduct

ProductKey (PK)
ProductID (business key)
SKU
ProductName
Category
Brand
UnitCost
StartDate
EndDate

fact.FactSales

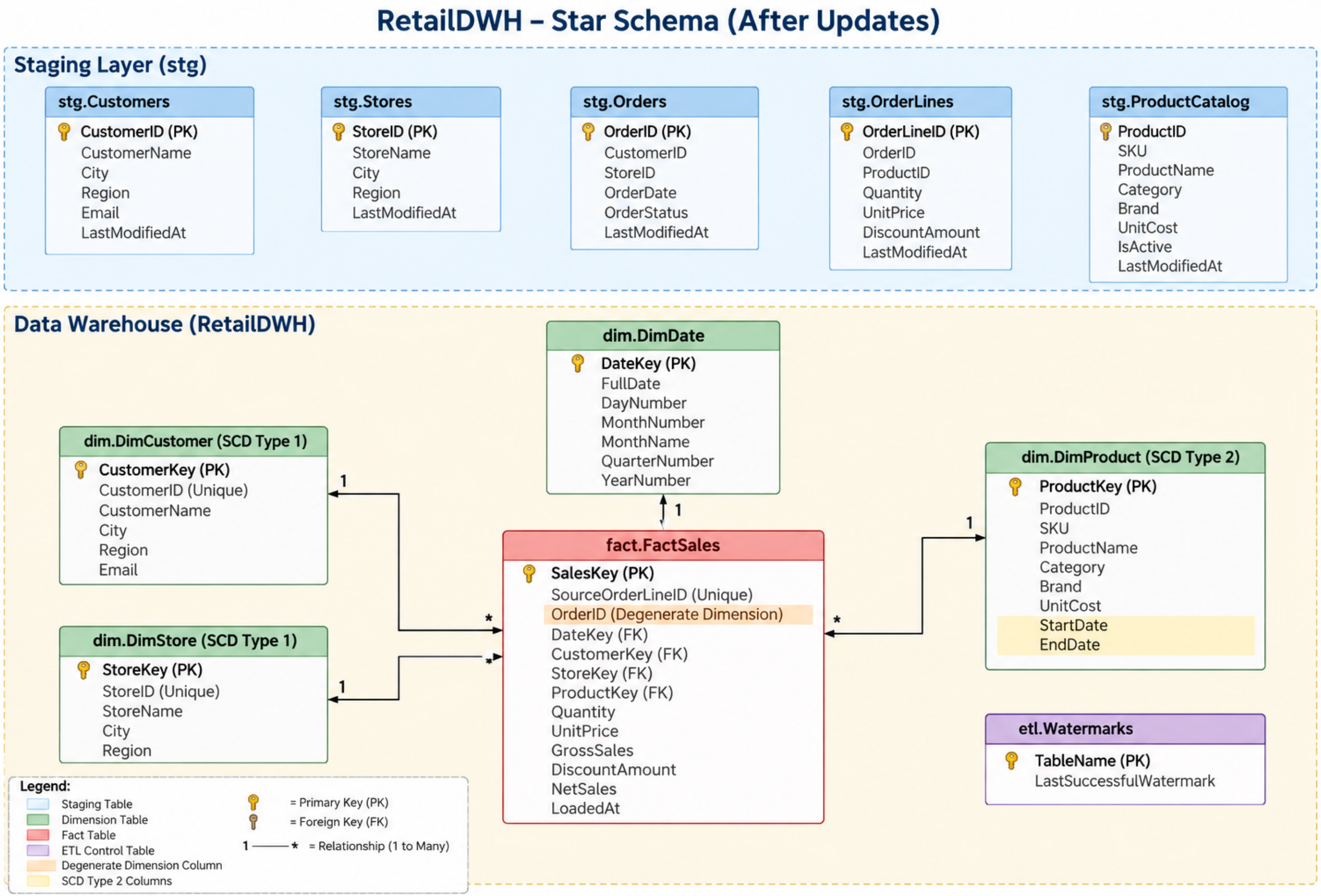
SalesKey (PK)
SourceOrderLineID (UNIQUE)
OrderID (DEGENERATE DIM)
DateKey (FK)
CustomerKey (FK)
StoreKey (FK)
ProductKey (FK)
Quantity
UnitPrice
GrossSales
DiscountAmount
NetSales
LoadedAt

WHY THE DWH HAS EXTRA COLUMNS

Source data contains operational IDs and transaction values. The DWH adds surrogate keys, derived calendar fields, SCD Type 2 validity dates, and calculated measures.

DEGENERATE DIMENSION: OrderID stays inside FactSales to group order lines; a separate DimOrder is unnecessary when the order number has no descriptive attributes.

RETAILDWH | STAR SCHEMA (SUPPLIED DIAGRAM)



DESIGN EXPLANATION

Why columns are added and why OrderID is degenerate

1. Why are some columns missing from the source?

The OLTP tables are built for daily transactions. The DWH is designed for historical analysis. It is normal for warehouse columns to be generated during ETL rather than copied from a source table.

DateKey, DayNumber, MonthNumber, QuarterNumber, YearNumber

Derived from Orders.OrderDate to build dim.DimDate.

CustomerKey, StoreKey, ProductKey

Warehouse surrogate keys generated for dimensions and referenced by FactSales.

StartDate and EndDate

Created by SCD Type 2 processing to preserve product versions. They are not required in staging.

GrossSales and NetSales

Calculated in the fact load: GrossSales = Quantity x UnitPrice; NetSales = GrossSales - DiscountAmount.

SalesKey and LoadedAt

Technical warehouse columns for row identity and load auditing.

2. Why is OrderID a Degenerate Dimension?

OrderID identifies the business transaction, but an order number alone has no descriptive attributes to justify a separate DimOrder table. Keep OrderID directly in fact.FactSales so several order lines can be grouped under the same order.

OrderID = 20001

OrderLineID 50001 -> FactSales row 1

OrderLineID 50002 -> FactSales row 2

Fact grain: one row per source OrderLineID. SourceOrderLineID prevents duplicate line loads. OrderID is repeated across those lines and is useful for order-level grouping. It is not a surrogate key or a foreign key to DimOrder.

3. What students must explain

- Why staging tables preserve source columns.
- Why dimension keys are different from source business IDs.
- Why DimProduct needs multiple historical versions (SCD Type 2).
- Why OrderID remains in FactSales rather than a separate dimension.
- Why FactSales must be loaded after the dimension keys are available.

03 | Create the staging layer

Create a new database named **RetailDWH**, then create schemas **stg**, **dim**, **fact** and optionally **etl**. Build the five staging tables below. The column lists reflect the supplied sources.

Staging table	Required source columns and SQL Server data types
stg.Customers	CustomerID INT; CustomerName NVARCHAR(100); City NVARCHAR(50); Region NVARCHAR(50); Email NVARCHAR(120); LastModifiedAt DATETIME2(3)
stg.Stores	StoreID INT; StoreName NVARCHAR(100); City NVARCHAR(50); Region NVARCHAR(50); LastModifiedAt DATETIME2(3)
stg.Orders	OrderID INT; CustomerID INT; StoreID INT; OrderDate DATETIME2(0); OrderStatus VARCHAR(20); LastModifiedAt DATETIME2(3)
stg.OrderLines	OrderLineID INT; OrderID INT; ProductID INT; Quantity INT; UnitPrice DECIMAL(12,2); DiscountAmount DECIMAL(12,2); LastModifiedAt DATETIME2(3)
stg.ProductCatalog	ProductID INT; SKU NVARCHAR(30); ProductName NVARCHAR(100); Category NVARCHAR(60); Brand NVARCHAR(60); UnitCost DECIMAL(12,2); IsActive CHAR(1); LastModifiedAt DATETIME2(3)

Staging requirements

• Preserve original business IDs and source timestamps. Choose NULL handling and data types intentionally.

• Optional: add **StagingRowID BIGINT IDENTITY(1,1)** to each table as a technical row identifier; it is **not** a dimension surrogate key.

• Do not add SCD StartDate or EndDate to ProductCatalog staging unless the source actually provides business-effective dates.

• For the first exercise, use a **full-refresh staging snapshot**. A production incremental approach is a separate design decision.

• Do not put a unique constraint on stg.ProductCatalog.ProductID if you want to detect duplicate rows from the file during validation.

Check: The operational SQL source has no product master table. Product attributes come from ProductCatalog_Batch1.csv and must enter the same staging area as the SQL tables.

04 | Design the dimensions

Create these tables under the **dim** schema. Use surrogate keys for Customer, Store and Product. This assignment uses **DateKey (YYYYMMDD)** as the DimDate primary key.

Dimension	Required columns / constraints	Change behavior
dim.DimDate	DateKey INT PK; FullDate DATE UNIQUE; DayNumber INT; MonthNumber INT; MonthName NVARCHAR(20); QuarterNumber INT; YearNumber INT	Calendar dimension; one row per date.
dim.DimCustomer	CustomerKey INT IDENTITY PK; CustomerID INT UNIQUE; CustomerName NVARCHAR(100); City NVARCHAR(50); Region NVARCHAR(50); Email NVARCHAR(120)	SCD Type 1: overwrite changed attributes.
dim.DimStore	StoreKey INT IDENTITY PK; StoreID INT UNIQUE; StoreName NVARCHAR(100); City NVARCHAR(50); Region NVARCHAR(50)	SCD Type 1: overwrite changed attributes.
dim.DimProduct	ProductKey INT IDENTITY PK; ProductID INT (not unique); SKU NVARCHAR(30); ProductName NVARCHAR(100); Category NVARCHAR(60); Brand NVARCHAR(60); UnitCost DECIMAL(12,2); StartDate DATETIME2(3); EndDate DATETIME2(3)	SCD Type 2: preserve historical versions.

Product SCD Type 2 - expected logic

- New ProductID: insert its first dimension version with a new ProductKey.
- Tracked attribute changed: close the current version and insert a new version with a new ProductKey.
- Use non-overlapping time intervals: **StartDate <= event time < EndDate**. Set the current row EndDate to a far-future value.
- Do not make ProductID unique in DimProduct; historical versions intentionally repeat it.
- Source **IsActive** means product business status; it is not the same as the SCD version-validity dates. If retained in DimProduct, define whether it is a tracked attribute.

Design decision: This lab intentionally uses SCD Type 2 for DimProduct, matching the historical-product exercise. The supplied OLTP and CSV files provide source data; they do not define your warehouse dimensions.

05 | Build fact.FactSales

The fact table must store **one row per source order line**. Use dimension keys, not CustomerID / StoreID / ProductID, as the four foreign keys.

Column	Suggested type	Role
SalesKey	BIGINT IDENTITY	Primary key for fact row
SourceOrderLineID	INT UNIQUE	Source line ID / duplicate protection
OrderID	INT	Degenerate dimension (order number)
DateKey	INT	FK to dim.DimDate(DateKey)
CustomerKey	INT	FK to dim.DimCustomer(CustomerKey)
StoreKey	INT	FK to dim.DimStore(StoreKey)
ProductKey	INT	FK to dim.DimProduct(ProductKey)
Quantity	INT	Additive quantity measure
UnitPrice	DECIMAL(12,2)	Price for the source order line
GrossSales	DECIMAL(18,2)	Calculated amount before discount
DiscountAmount	DECIMAL(18,2)	Discount from source line
NetSales	DECIMAL(18,2)	Calculated amount after discount
LoadedAt	DATETIME2(3)	Warehouse load timestamp

Business rules students must document

- Only **Completed** orders contribute to FactSales; do not count Pending or Cancelled orders as completed revenue.
- **GrossSales = Quantity x UnitPrice**; **NetSales = GrossSales - DiscountAmount**.
- Find CustomerKey and StoreKey using source business IDs; resolve DateKey from the order date.
- For DimProduct SCD Type 2, resolve the ProductKey whose validity interval contains the order event date/time (according to your documented effective-date policy).
- A source order line with an unknown product must be rejected/logged or handled with a documented Unknown member policy - never silently assign a random ProductKey.

06 | Implementation steps and submission

Work in the sequence below. Students should be able to show the created SQL objects and explain each modeling decision without relying on a prebuilt DWH script.

Step	Task	Evidence to submit
01	Execute the OLTP create script, then sample-data script.	Screenshot or query showing source tables.
02	Create RetailDWH and its schemas.	Screenshot of stg, dim and fact schemas.
03	Create all five staging tables with source-compatible types.	DDL script and table definitions.
04	Create DimDate, DimCustomer, DimStore and DimProduct.	DDL with keys and SCD2 dates.
05	Create FactSales and four foreign-key constraints.	DDL and relationship diagram.
06	Explain how SSIS loads staging, dimensions, then fact.	Short package/control-flow sketch.
07	Write data-quality checks and reporting SQL.	Validation SQL and sample results.

Acceptance checklist

- Five staging tables exist under **stg**; four dimensions under **dim**; one fact under **fact**.
- The fact has four valid foreign keys and a documented grain.
- The DimProduct design allows more than one historical version per ProductID.
- Students can explain **ProductID vs ProductKey**, **OrderID as a degenerate dimension**, and **1-to-many dimension relationships**.
- The provided sample includes **OrderLineID 50470 / ProductID 9999**, which does not exist in Batch 1. Describe how it will be handled.
- No source SQL file should be edited to fake missing product records.

Suggested validation questions

How many source orders have status Completed? What is the number of valid order lines after product lookup? Does **COUNT(DISTINCT SourceOrderLineID)** equal the fact row count? Can you calculate monthly revenue by brand and store?

Final deliverables: (1) RetailDWH DDL .sql file, (2) screenshots of SSMS tables and relationships, (3) one-page explanation of grain, keys and SCD choices, (4) validation SQL, and (5) optional SSIS .dtsx packages.